Solution: Minimum Add to Make Parentheses Valid

Problem Statement

Given a string str containing '(' and ')' characters, find the minimum number of parentheses that need to be added to a string of parentheses to make it valid.

A valid string of parentheses is one where each opening parenthesis '(' has a corresponding closing parenthesis ')' and vice versa. The goal is to determine the least amount of additions needed to achieve this balance.

Examples

1. **Example 1**:
   - **Input**: "(()"
   - **Expected Output**: 1
   - **Justification**: The string has two opening parentheses and one closing parenthesis. Adding one closing parenthesis at the end will balance it.
2. **Example 2**:
   - **Input**: "))(("
   - **Expected Output**: 4
   - **Justification**: There are two closing parentheses at the beginning and two opening at the end. We need two opening parentheses before the first closing and two closing parentheses after the last opening to balance the string.
3. **Example 3**:
   - **Input**: "(()())("
   - **Expected Output**: 1
   - **Justification**: The string has three opening parentheses and three closing parentheses, with an additional opening parenthesis at the end. Adding one closing parenthesis at the end will balance it.

Solution

To solve this problem, we track the balance of parentheses as we iterate through the string. We initialize two counters: one for the balance of parentheses and another for the count of additions needed.

For each character in the string, if it's an opening parenthesis '(', we increase the balance. If it's a closing parenthesis ')', we decrease the balance. If the balance is negative at any point (which means there are more closing parentheses than opening ones), we increment the additions counter and reset the balance to zero.

The total number of additions required is the sum of the additions counter and the absolute value of the final balance, ensuring that all unmatched opening parentheses are also accounted for. This approach efficiently computes the minimum number of parentheses to be added for the string to become valid.

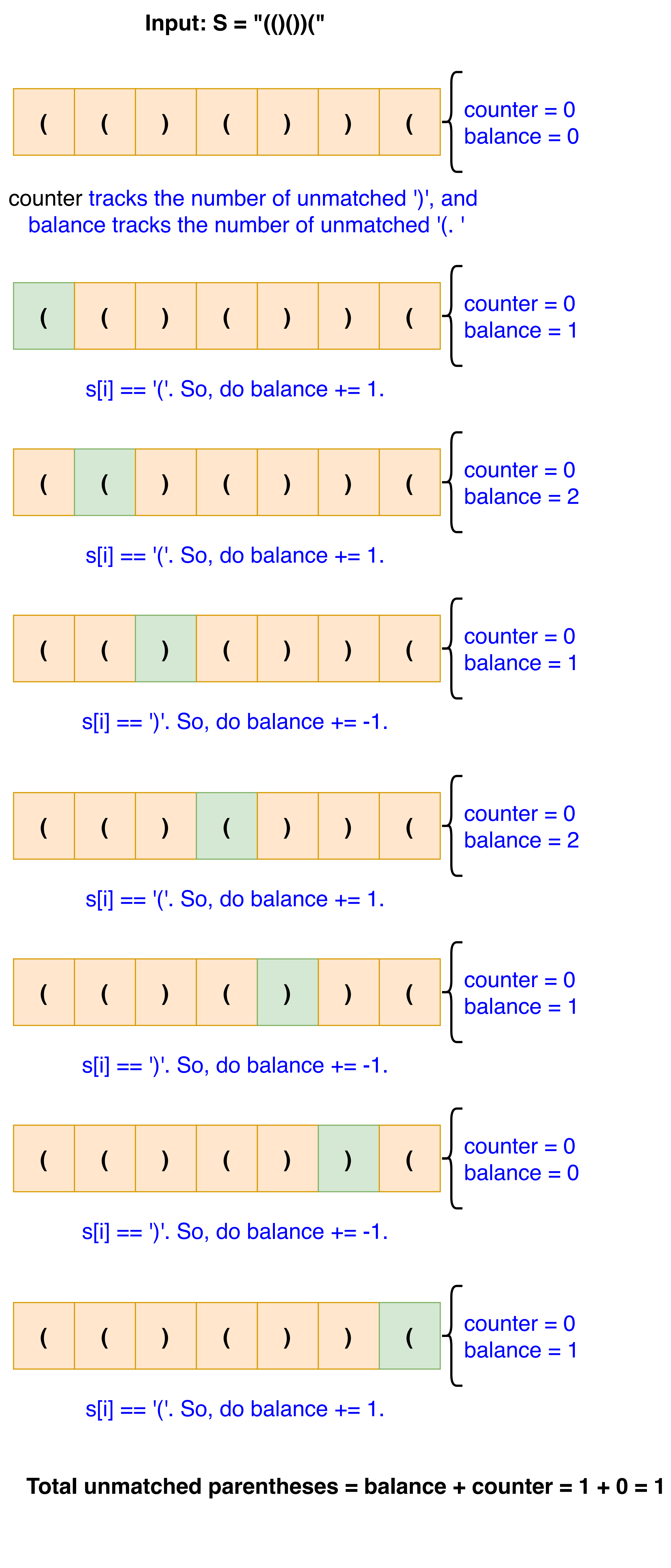
1. **Initialization**: Start with a counter set to zero, representing the number of parentheses needed to balance the string.
2. **Iterating through the String**: For each character in the string, determine if it's an opening or closing parenthesis.
3. **Handling Opening Parenthesis**: Increment the balance counter for each opening parenthesis, indicating a pending closing parenthesis is needed.
4. **Handling Closing Parenthesis**: For each closing parenthesis, if there is an unmatched opening parenthesis (balance counter > 0), decrement the balance. If not, increment the counter, indicating an additional opening parenthesis is needed.
5. **Completion**: The final value of the counter represents the total number of additional parentheses required to balance the string.

Algorithm Walkthrough

Let's apply the algorithm to the input string "(()())(":

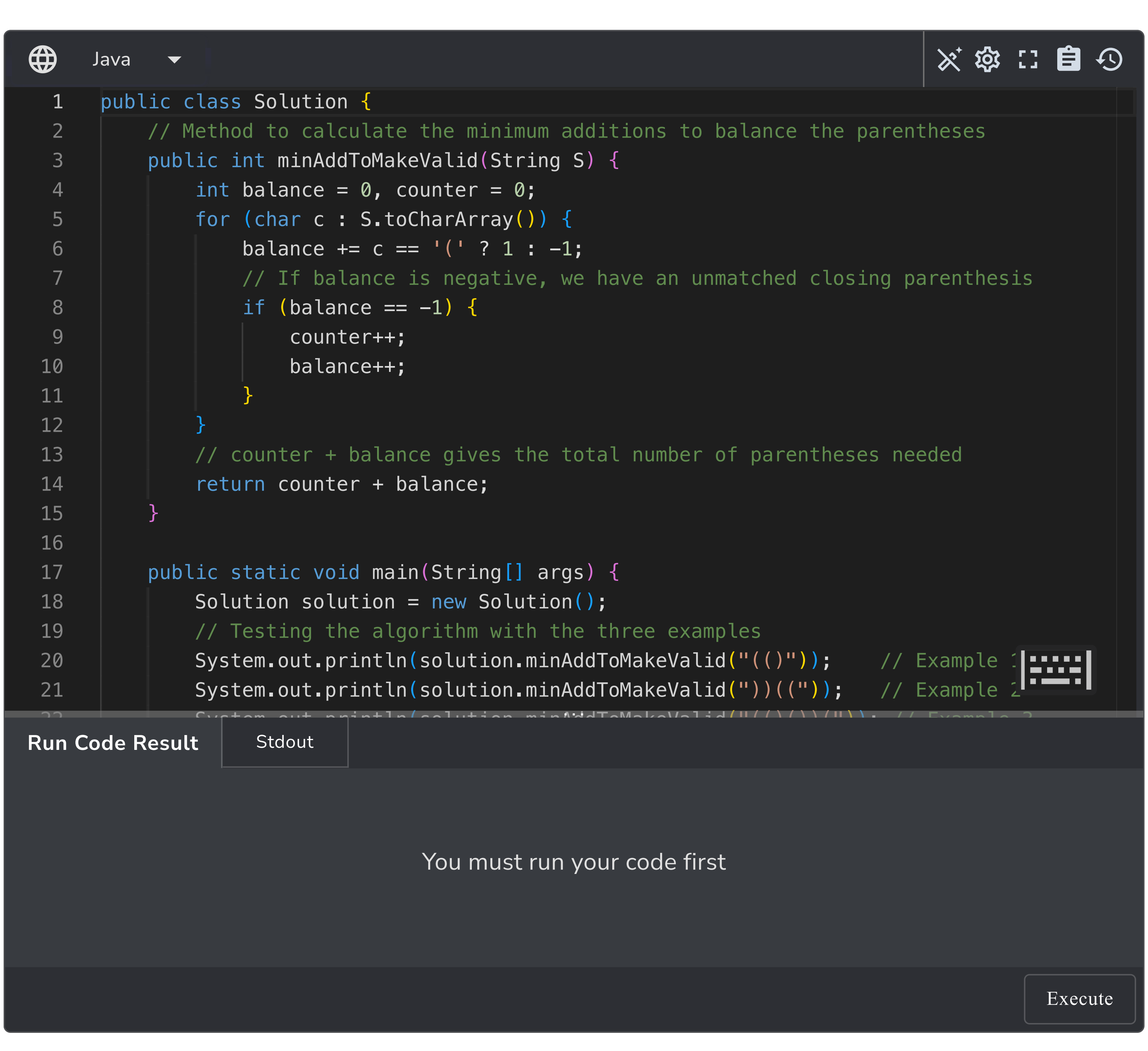
1. **Initialize Variables**:
   - `balance = 0`
   - `counter = 0`
2. **Iterate Through the String** "(()())(":
   - **First Character** '(' :
     - Increment `balance` → `balance = 1` (1 unmatched opening parenthesis).
   - **Second Character** '(' :
     - Increment `balance` → `balance = 2` (2 unmatched opening parentheses).
   - **Third Character** ')' :
     - Decrement `balance` → `balance = 1` (1 unmatched opening parenthesis).
   - **Fourth Character** '(' :
     - Increment `balance` → `balance = 2` (2 unmatched opening parentheses).
   - **Fifth Character** ')' :
     - Decrement `balance` → `balance = 1` (1 unmatched opening parenthesis).
   - **Sixth Character** ')' :
     - Decrement `balance` → `balance = 0` (all parentheses matched so far).
   - **Seventh Character** '(' :
     - Increment `balance` → `balance = 1` (1 unmatched opening parenthesis).
3. **Final Calculation**:
   - At the end of the string, `balance = 1` and `counter = 0`.
   - Add `counter` and `balance` → `0 + 1 = 1`.
4. **Return Result**:
   - The final result is `1`, indicating that 1 additional closing parenthesis is required to make the string "(()())(" valid.

This walkthrough demonstrates that to balance the given string "(()())(", we need to add just one closing parenthesis.


Input: S = "(()())("
(( ) ( )) (
counter = 0
balance = 0
counter tracks the number of unmatched ')', and balance tracks the number of unmatched '(. '
counter = 0
balance = 1
s[i] == '('. So, do balance += 1.
counter = 0
balance = 2
s[i] == '('. So, do balance += 1.
counter = 0
balance = 1
s[i] == ')'. So, do balance += -1.
counter = 0
balance = 2
s[i] == '('. So, do balance += 1.
counter = 0
balance = 1
s[i] == ')'. So, do balance += -1.
counter = 0
balance = 0
s[i] == ')'. So, do balance += -1.
counter = 0
balance = 1
s[i] == '('. So, do balance += 1.
Total unmatched parentheses = balance + counter = 1 + 0 = 1

Code

Here is the code for this algorithm:



```java
public class Solution {
    // Method to calculate the minimum additions to balance the parentheses
    public int minAddToMakeValid(String S) {
        int balance = 0, counter = 0;
        for (char c : S.toCharArray()) {
            balance += c == '(' ? 1 : -1;
            // If balance is negative, we have an unmatched closing parenthesis
            if (balance == -1) {
                counter++;
                balance++;
            }
        }
        // counter + balance gives the total number of parentheses needed
        return counter + balance;
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
        // Testing the algorithm with the three examples
        System.out.println(solution.minAddToMakeValid("(()"));    // Example 1
        System.out.println(solution.minAddToMakeValid("))((")); // Example 2
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The time complexity of this algorithm is O(n), where n is the length of the string. This is because we iterate through the string once.
- **Space Complexity**: The space complexity is O(1), as we only use a fixed amount of additional space (counter and balance variables) regardless of the input size.

← Previous
Minimum Add To Make Parentheses Valid (Medium)

Next →
Remove Duplicate Letters (Medium)

☐ Mark as Completed